

ACCESO A DATOS

U.T. 1 – MANEJO DE FICHEROS

Jose Medina Velasco

Índice

- Gestión de ficheros y directorios
- Gestión de flujos de datos desde/hacia ficheros
 - Basados en bytes
 - Basados en caracteres
- Excepciones
- Trabajo con ficheros XML
- Trabajo con ficheros JSON

Gestión de ficheros y directorios

¿Qué es un fichero?

Un fichero es una **colección ordenada de datos** que tienen entre sí una relación y que se almacenan de forma **permanente** en un dispositivo de memoria **no volátil**.

Gestión de ficheros y directorios

Tipos de ficheros

Los ficheros siempre siguen un formato dado, que define su contenido y estructura, elegido por el desarrollador de una aplicación.

Podemos clasificar los ficheros en:

- **Ficheros binarios:** Contienen representaciones exactas del contenido binario de los datos, por lo que no son editables de forma directa.
- **Ficheros de texto:** Contienen datos representados por caracteres alfanuméricos que pueden ser leídos y modificados a través de un editor de texto.

Gestión de ficheros y directorios

¿Qué son los directorios?

Un directorio o carpeta es un **tipo** exclusivo **de archivo** que sólo **contiene la información** necesaria **para acceder a archivos o a otros directorios**.

Los sistemas de archivos (abstracciones del hardware de almacenamiento que suele proporcionar el sistema operativo) suelen representarse como un árbol invertido de directorios.

Los directorios se ramifican en sentido descendente desde el directorio raíz del diagrama en árbol y pueden contener archivos y subdirectorios.

Gestión de ficheros y directorios

La gestión de ficheros y directorios se puede hacer normalmente usando las interfaces gráficas que proporcionan los entornos de escritorio de los S.O., aunque también tenemos la posibilidad de hacerlo de forma programática mediante lenguajes de programación que acceden a las APIs de sistema.

Gestión de ficheros y directorios

En los lenguajes de programación orientados a objetos (POO) se suele hacer uso de clases específicas para llevar a cabo la creación, escritura, lectura y borrado de los archivos y directorios.

Gestión de ficheros y directorios

En Python, no hacen falta librerías externas para abrir, cerrar, leer o escribir en ficheros. Se puede usar de forma nativa la función `open()`.

Gestión de ficheros y directorios

Abrir un fichero

Para abrir un fichero usamos la función `open()`, pero tiene varios modos diferentes:

- „**r**“: Lectura (por defecto), Error si no existe
- „**w**“: Escritura (sobreescribe si existe)
- „**a**“: Añadir al final (crea si no existe)
- „**x**“: Crear y escribir (error si ya existe)
- „**r+**“: Lectura y escritura
- „**b**“: Modo binario (se combina „rb“ y „wb“...)
- „**t**“: Modo texto (por defecto)

Gestión de ficheros y directorios

Forma recomendada de abrir y cerrar un fichero

Para no dejar abierto un fichero, lo que se usa es abrirlo en un ámbito muy cerrado, utilizando la palabra clave „**with**“, que lo cierra automáticamente al terminar el bloque:

```
with open("datos.txt", "r", encoding="utf-8") as f:  
    contenido = f.read()           # Lee todo el fichero
```

Gestión de ficheros y directorios

Forma recomendada de abrir y cerrar un fichero

With garantiza el cierre del fichero aunque ocurra alguna excepción.

Podemos abrir más de un fichero a la vez en el mismo bloque with:

```
with open("lectura.txt", "r", encoding="utf-8") as l, open("lectura_2.txt", "r", encoding="utf-8") as l2:  
    print(l.read())  
    print(l2.read())
```

Gestión de ficheros y directorios

Forma clásica de abrir y cerrar un fichero

La forma clásica obliga a cerrar el fichero de forma manual, capturando una excepción:

```
f = open("datos.txt", "r", encoding="utf-8")
try:
    contenido = f.read()
finally:
    f.close()    # Importante cerrar siempre
```

Gestión de ficheros y directorios

Métodos de lectura de texto

Tenemos distintos métodos de lectura dependiendo de nuestro interés:

- **f.read()**: Lee todo el contenido como str.
- **f.read(n)**: Lee n caracteres.
- **f.readline()**: Lee una línea.
- **f.readlines()**: Devuelve una lista con todas las líneas de un archivo de texto.
- **for linea in f**: Itera línea a línea el archivo.

Tienen un `\n`
al final

Gestión de ficheros y directorios

Métodos de escritura de texto

Hay que especificar el modo, „w“ sobrescribe y „a“ añade. Importante especificar el encoding („utf-8“).

- **f.write(texto)**: Escribe el texto en buffer
- **f.writelines(lineas)**: Escribe las líneas de un iterable (lista, por ejemplo) en buffer.

⚠ Por defecto, no se escribe en el fichero, para ello hay que hacer `flush()`, o utilizar `with`. Cuando se escribe dentro de un bloque `with`, al terminar este se guarda en un fichero.

Gestión de ficheros y directorios

Métodos de lectura binaria

Tenemos distintos métodos de lectura dependiendo de nuestro interés:

- **f.read()**: Lee todo el contenido como lista de bytes.
- **f.read(n)**: Lee n bytes. Se usa para leer por „chunks“ o trozos

```
with open("archivo.bin", "rb") as f:
    while True:
        trozo = f.read(4096) # Lee 4 KB cada vez
        if not trozo:       # Fin del archivo
            break
        # procesar trozo
```

Gestión de ficheros y directorios

Métodos de escritura binaria

Los archivos binarios se suelen escribir en trozos, usando chunks.

```
with open("lectura.txt", "rb") as l, open("copia.txt", "wb") as e:  
    chunk = l.read(2048)  
    while chunk:  
        e.write(chunk)  
        chunk = l.read(2048)
```

Gestión de ficheros y directorios

Utilidades posibles de la lectura y escritura de archivos binarios

- 1) Copiar ficheros sin modificarlos
- 2) Comprobar el tipo de archivo por su cabecera (primeros bytes)
- 3) Guardar (crear el fichero con información) si de otra fuente nos envían bytes.
- 4) Leer registros de tamaño fijo (bajo nivel) (por ejemplo, leer 4 bytes e interpretarlos como un entero, se usa struct)

Gestión de ficheros y directorios

Gestión de ficheros y directorios

Hay tres librerías principales (os/os.path, pathlib, además de shutil). pathlib es la más moderna, orientada a objetos, mientras la otra es estructurada.

```
from pathlib import Path
```

pathlib

```
p = Path('docs/informe.txt')
p.exists() p.is_file()
p.parent p.name p.suffix
p.read_text(encoding='utf-8')
p.write_text('hola')
p.mkdir(parents=True)
list(p.glob('*.py'))
p.rename() p.unlink() p.rmdir()
archivo = p / 'informe.pdf'
archivo = p.joinPath('informe.pdf')
```

```
import shutil
```

shutil

```
shutil.copy()
shutil.copytree()
shutil.rmtree()
shutil.move()
```

Gestión de ficheros y directorios

XML

XML viene de las siglas Extensible Markup Language (Lenguaje de Marcas eXtensible), ya que permite crear las etiquetas que nosotros necesitemos en nuestros proyectos, sin tener que adecuarse a las estándar de HTML, por ejemplo.

Gestión de ficheros y directorios

XML

XML es un formato de datos estructurado, para poder usarlo en Java, tendremos que transformarlo en una estructura de datos que se pueda manipular (analizador o parser), o directamente mapeando los datos a objetos („binding“).

Hay diferentes modelos de parser, basados en árboles (DOM), basados en eventos (SAX), etc., cada uno tiene ventajas e inconvenientes según varios criterios.

Gestión de ficheros y directorios

XML (DOM)

Carga todo el documento XML en memoria como un árbol de nodos.

Ventajas:

- Fácil de navegar, modificar y consultar
- Bueno para documentos pequeños o para accesos repetidos.

Desventajas:

- Consume bastante memoria si el XML es grande
- Puede ser lento en inicializar

Gestión de ficheros y directorios

XML (SAX)

Va leyendo el XML de forma secuencial („streaming“).

Ventajas:

- Es muy rápido porque carga solo lo siguiente en el árbol
- Bueno para leer y procesar los datos.

Desventajas:

- No mantiene en memoria todo.
- Código complejo
- No pensado para modificar el XML

Gestión de ficheros y directorios

En python podemos utilizar las siguientes librerías:

- `lxml.etree`: para uso DOM
- `xml.sax`: para uso SAX

Nosotros vamos a utilizar `lxml`. Esta librería soporta Xpath, XSLT, validación, etc.

Gestión de ficheros y directorios

La instalación de lxml se hace con:

```
pip install lxml
```

Es posible que necesitemos hacer un entorno virtual, con
`python -m venv entorno`

Gestión de ficheros y directorios

Parseo de ficheros XML

Si tenemos un fichero XML que queremos trabajar, lo cargamos de esta manera:

```
from lxml import etree  
root = etree.parse("papers_arxiv.xml").getroot()
```

Gestión de ficheros y directorios

Parseo de ficheros XML

Podemos recorrer todo el árbol de esta manera:

```
def imprimir_arbol(elemento, nivel=0):  
    indent = "\t" * nivel  
    attrs = " ".join(f'{k}="{v}"' for k, v in elemento.attrib.items())  
    texto = (elemento.text or "").strip()  
    print(f"{indent}<{elemento.tag} {attrs}> {texto}")  
    for hijo in elemento:  
        imprimir_arbol(hijo, nivel + 1)
```

FIN